

Chapter 1

Instruction Set Architecture

(1 Q) (7 marks)

(Not asked yet)

1. What is a programming language? What are the levels of programming languages?

- A programming language is a computer language that programmers use to communicate with computers.
- It is a set of instructions written in any specific language to perform a specific task.
- Example: C, C++, Java, Python, etc.

The levels of programming languages are:

1. Machine-level language

- A computer program written in machine code is executed directly by the computer's central processing unit.
- It uses binary numbers, 0's and 1's.
- It doesn't require a translator and is directly understood by the computer.
- It is faster and more efficient than other languages.
- People cannot understand it.

2. Assembly level language

- It uses mnemonics, a syntax similar to the English language that people can understand.
- It allows programmers to control hardware directly, making it useful for system-level programming.
- It is used in applications such as real-time systems and micro-controller-based embedded systems.
- It is memory efficient as it requires less memory.
- It is faster in speed.
- It requires an assembler to convert the source code to machine code.

3. High-level programming language

- It is a programmer-friendly language that is very easy to understand and also very simple to debug and maintain.
- They offer built-in functions and features that help programmers work with complex tasks without worrying about low-level details. i.e., they offer abstraction.
- It needs a compiler to translate source code into machine code.
- They are portable and can run on different types of computers with little adjustment.
- It is less memory efficient.
- It is slower than machine-level language and assembly-level language.
- Example: C, C++, Java, etc.

2. Define computer organization and architecture.

- **Computer Organization** refers to how the hardware components of a computer system, such as the memory, central processing unit (CPU), and input/output devices, are interconnected and structured to work together.
- It involves designing the data paths and control units that facilitate data movement and manipulation within the system.
- **Computer Architecture** involves higher-level design decisions that define the capabilities and performance of a computer system.
- It encompasses the instruction set architecture (ISA), which specifies the set of machine-level instructions the CPU can execute.
- Architectural choices affect the computer's speed and efficiency, such as memory hierarchy design for data storage, pipelining to optimize task execution, and parallelism to perform multiple tasks simultaneously.

(Asked 6 times)

3. Explain the compiling and assembling process with suitable examples.

OR

Describe the assembly process for assembly language programs.
How does it differ from the compilation process.?

Compiling

- The compilation is the process of converting the source code written in a high-level programming language to an object code.
- The compilation process makes sure that the source code follows syntax.
- It converts the whole source code into machine code at once.
- It takes less execution time compared to an assembler.
- The steps involved are:
 1. Programmers write code using a high-level programming language like C, C++, or Java.
 2. The compiler is like a translator. It takes the high-level code and translates it into a lower-level code that the computer can understand and execute.
 3. The compiler performs several tasks like Lexical Analysis, Syntax Analysis, and Semantic Analysis.
 4. The compiler might make the code faster or use less memory by optimizing it.
 5. The compiler generates the object file i.e., machine code—the binary instructions that the computer's CPU can execute.
 6. If the program is composed of multiple source files, the linker combines the object files and generates the final executable file.

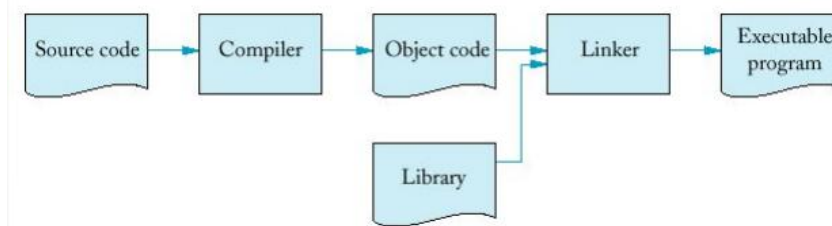


Fig: Compilation Process

Assembling Process:

- Assembling is the process of converting an assembly code into object code (machine code).
- An assembler is slower than a compiler because it does not convert the whole program at once, it does it one by one.

The steps involved are:

1. Programmers write code using Assembly Language, which is a low-level language close to the machine code.
2. The assembler takes the Assembly Language code and translates it directly into machine code.
3. The assembler converts the human-readable mnemonic instructions (like ADD, MOV) into their binary equivalents.
4. The assembler produces object files containing the machine code and memory addresses.
5. For programs with multiple parts, a linker might combine different object files into one executable file.

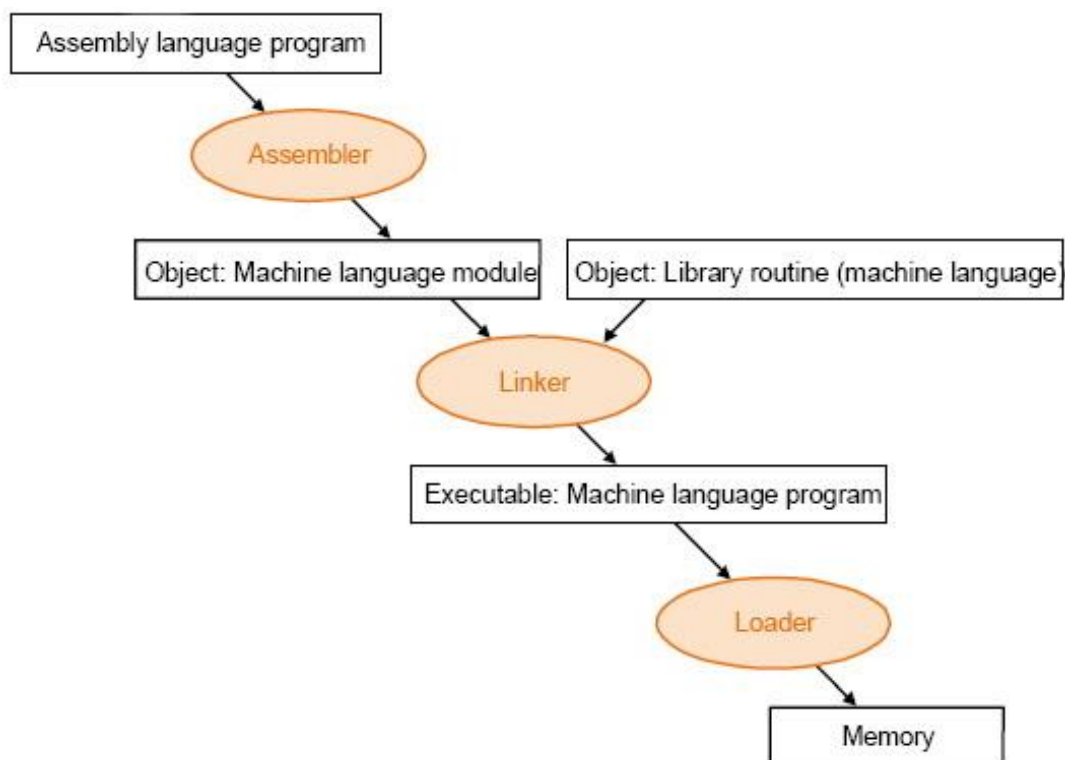


Fig: Assembling process

(Asked 3 times)

3. Describe the different types of instruction with examples.

(Instruction Types)



Instructions are the commands that a computer's central processing unit (CPU) can understand and execute.

- They tell the computer what operations to perform on data.
- There are three types of instructions:

1. Data Transfer Instructions:

- Data transfer instructions are like moving data around in a computer's memory.
- They do not modify data; they only copy a value to the destination.
- Example:
 - **Load:** This instruction brings data from memory into a CPU register.
 - **Store:** Store does the opposite—it takes data from a CPU register and puts it into memory.
 - **Move:** Move copies data from one place to another within memory or between memory and registers.

2. Data Operation Instructions:

- Data operation instructions perform calculations or logical operations on data:
- They modify the data.
- Example:
 - **Add:** Adds two numbers together, just like you'd add numbers on paper.
 - **Subtract:** Subtracts one number from another.
 - **Multiply:** Multiplies two numbers together.
 - **Divide:** Divides one number by another.

3. Program Control Instructions:

- Program control instructions manage the flow of execution of program.
- Example:
 - **Jump:** Jump instructions tell the computer to go to a different part of the program. It's like skipping to another page in a book.
 - **Call:** Call instructions are like making a phone call. They tell the computer to go to another section of the program, perform a task, and then come back.
 - **Return:** Return instructions are used with call instructions. They bring the computer back to where it left off after a task is done.

(Asked 2 times)

4. Explain data types.

- Data types define the kind of information that can be stored and manipulated by a program.
- Different types of data are stored differently in a computer's memory.
- Here are common data types:
 1. **Integer:** Integers are whole numbers without decimal points. They can be positive or negative. For example, 5, -10, and 0 are integers.
 2. **Floating-Point:** Floating-point numbers represent real numbers with decimal points. Examples include 3.14 and -0.002.
 3. **Character:** Characters represent individual letters, numbers, or symbols. For example, 'A', '7', and '&' are characters.
 4. **String:** A string is a sequence of characters, like a word or a sentence. It's used to represent text. For instance, "Hello, World!" is a string.
 5. **Boolean:** Booleans have two values: true or false. They're used for making logical decisions in a program.
 6. **Array:** An array is a collection of elements of the same data type. It's like a list that holds multiple pieces of data under one name.

(Asked 3 times)

5. Explain instruction format in detail.

OR

Describe the different types of instruction format.

- Instruction format refers to the structure and layout that computer instructions follow in machine code.
- It defines how the various components of an instruction, such as the operation code (opcode) and operands, are arranged.
- The instruction format provides a standardized way for the computer's hardware to interpret and execute commands.
- The different types of instruction formats are:

1. Zero Address Instruction Format:

- In a zero-address instruction format, the command itself contains all the information the computer needs to perform a specific operation.
- These instructions are often used for operations that the computer's hardware already understands really well, like basic arithmetic.
- **Example:** The instruction "ADD" in a zero address format directly tells the computer to add two numbers that are already in specific memory locations. (PUSH A, PUSH B and ADD will perform addition of two values at the top of stack)

2. One Address Instruction Format:

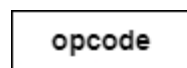
- In a one-address instruction format, you're telling the computer to perform an operation using one piece of data.
- You still need to specify what operation to do, but the data is already known from a specific place.
- **Example:** The instruction "INCREMENT A" in a one address format tells the computer to increase the value stored in memory location A by 1.

3. Two Address Instruction Format:

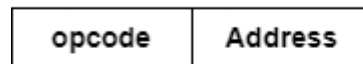
- In a two-address instruction format, you're telling the computer to perform an operation using data from two specific places.
- You mention both the operation and where the data comes from.
- Example: The instruction "ADD A, B" in a two-address format means add the values stored in memory locations A and B, and store the result in A.

4. Three Address Instruction Format:

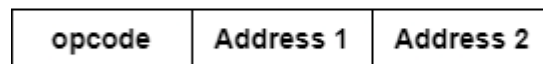
- In a three-address instruction format, you're providing the computer with all the details it needs: the operation and the exact locations of the data.
- Example: The instruction "C = A + B" in a three address format means add the values from memory locations A and B, and store the result in memory location C.



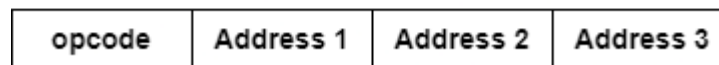
Zero address instruction format



One address instruction format



Two address instruction format



Three address instruction format

Fig: Instruction Format

(Asked 5 times)

6. Define addressing modes. Explain the different types of addressing modes with examples.

OR

Explain different types of addressing modes along with their usages and applications.

- Addressing modes define how the operands (data) for instructions are specified and accessed in a computer's memory or registers.
- They provide rules for the CPU to find, retrieve, and manipulate data during program execution.
- The different types of addressing modes are:

1. Direct Addressing:

- **Working:** The instruction directly specifies where the data is in memory.
- **Usage:** When you know exactly where the data is located.
- **Application:** Accessing constant values, fixed data structures, or device registers.
- **Example:** LDAC 5, reads data from memory location 5 and stores it in the accumulator.

2. Indirect Addressing:

- **Working:** The address in the instruction is not the address of the operand, rather it contains the address of the memory location where the operand resides.
- **Usage:** When the address of the data is stored somewhere else.
- **Application:** Implementing pointers
- **Example:** LDAC @5 or LDAC (5), first retrieves the content of memory location 5, say 10. The CPU then retrieves the content of memory location 10, that actually contains the operand.

3. Register Direct Addressing:

- **Working:** The operand is present directly in a CPU register specified in the instruction.
- **Usage:** Efficient for quick operations involving data within registers.
- **Application:** Mathematical calculations, temporary storage, and comparisons.
- **Example:** LDAC R instruction gets data from register R.

4. Register Indirect Addressing:

- **Working:** The instruction refers to a register whose content holds the memory address of the operand. The CPU fetches the address from the register and then retrieves the data from that memory location.
- **Usage:** When you want to access data from memory locations stored in registers.
- **Application:** Looping through arrays, working with data structures.
- **Example:** LDAC R instruction gets the address from register R and then gets data from that address.

5. Immediate Addressing:

- **Working:** The operand value is directly given in the instruction itself.
- **Usage:** For quick calculations using fixed values or constants.
- **Application:** Arithmetic operations, comparisons, and simple calculations.
- **Example:** LDAC #5 instruction moves the data value 5 into the accumulator.

6. Implicit Addressing:

- **Working:** The instruction implies that the operation involves specific registers or predefined memory locations.
- **Usage:** When you're working with common registers or flags.
- **Application:** Accumulators, status flags, stack pointers.
- **Example:** CLAC, which clears the accumulator.

7. Relative Addressing:

- **Working:** Here the operand supplied is an offset. It is added to the content of the CPU Program Counter (PC) to generate the required address.
- **Usage:** For branching or looping based on calculated addresses.
- **Application:** Conditional jumps, loops, and program flow control.

- **Example:** LDAC \$5, It adds 5 to the address of the next instruction and stores value into the accumulator. If the current instruction is at location 10 and the next is at location 12, then the above instruction reads data from location 17 ($12+5$) and stores it in the accumulator.

8. Indexed Addressing:

- **Working:** It is the same as relative addressing mode but uses an Index Register instead of a PC. An offset is added to an Index(X) register address to calculate the operand's memory location.
- **Usage:** When accessing elements within arrays, tables, or data structures.
- **Application:** Looping through arrays, iterating over collections, working with matrices.
- **Example:** LDAC 5(X), If the value of the Index register is 10 then it reads data from location 15 ($5+10$) and stores it in the accumulator.

9. Base Addressing:

- **Working:** Similar to indexed addressing but uses a base register for calculation. It specifies an offset to be added to the contents of the base address register.
- **Usage:** For accessing data fields in records or structures using a fixed offset from a changing base address.
- **Application:** Handling records, structures, and complex data types.
- **Example:** LDAC 5(X), if the value of the base address register is 10 then it reads data from location 15 ($5+10$) and stores it in the accumulator.

(Asked 6 times)

7. Describe Instruction Set Architecture (ISA).

- Instruction Set Architecture (ISA) refers to the predefined set of instructions that a CPU can execute.
- ISA defines the types of operations a CPU can carry out, the format of instructions, and how they interact with memory and registers.
- ISA is crucial because it defines the capabilities and limitations of a computer's hardware.
- Different computer architectures have different ISAs, and they determine how software is written and how efficiently it can be executed on a particular hardware platform.
- For example, an ISA might include instructions like "ADD," "SUBTRACT," "LOAD," and "STORE."

- Programmers use these instructions to create software.
- ISA is very important in the design of microprocessors.
- Poorly designed ISA, even if implemented well leads to bad microprocessor design.
- Well-designed ISA leads to powerful microprocessors.
- No magical formula for designing ISA; same requirement can have different ISA design, each of which can be valid.
- Must evaluate trade-offs between performance, size and cost.

(Asked 7 times)

8. What are the considerations to be made while designing ISA.

OR

What are the major issues to be considered while designing an Instruction Set Architecture?

1. Orthogonality:

- Orthogonality means that instructions and addressing modes work independently of each other.
- Having an orthogonal ISA is generally considered a positive feature as it provides simplicity, efficiency, consistency, etc.
- However, achieving perfect orthogonality might not always be practical.
- Trade-offs may arise between orthogonality and hardware complexity.

2. Trade-offs:

- Designing an ISA involves balancing various factors, such as hardware complexity and performance.
- Managing trade-offs ensures that the architecture is efficient and practical while avoiding unnecessary complications.

3. Optimization of Registers:

- Deciding on the number and types of registers impacts how quickly data can be accessed and manipulated.
- Optimizing registers contributes to the efficiency and speed of instruction execution.

4. Data Type and Size:

- Specifying supported data types and sizes (e.g., integers, floats, byte lengths) ensures proper data handling.
- This ensures that the architecture can handle different types of data and operations accurately and efficiently.

5. Use of Conditional Instructions:

- Conditional instructions allow instructions to change behavior based on specific conditions, enhancing flexibility.
- They are crucial for branching and decision-making in programs, enabling dynamic control flow.

6. Interrupt Handling:

- Deciding how interrupts (external events that pause execution) are managed is vital for real-time systems and multitasking.
- Interrupt handling ensures the CPU can respond to external events promptly and efficiently.

7. Backward Compatibility:

- Backward compatibility ensures that software written for older versions of the architecture can still run on newer hardware.
- Compatibility prevents disruption during upgrades and protects investments made in existing software.
- If ISA of Pentium III were not backward compatible with ISA of Pentium II, it would not be able to run code written in Pentium II.

8. Energy Efficiency:

- Designing instructions that minimize power consumption contributes to longer battery life and environmentally friendly computing.
- Energy efficiency is increasingly important for mobile devices and environmentally conscious computing.

9. Parallelism and Pipelining:

- Supporting parallelism allows the CPU to perform multiple tasks simultaneously, boosting overall performance.
- Pipelining enhances throughput by breaking down instructions into stages, allowing concurrent execution.

Chapter 2

Computer Organization

(2Q) (15 marks)

(Asked 2 times)

1. Explain the basic computer organization with the help of a block diagram.

- The basic computer organization has three main components:
 - a. The CPU
 - CPU performs various operations and controls the computer.
 - It basically performs four operations, and they are fetch, decode, execute and store.
 - b. The memory sub-system
 - Memory sub-system is used to store programs being executed by the CPU along with the necessary data.
 - It can consist of RAM and ROM.
 - c. I/O sub-system
 - The I/O sub-system allows the CPU to interact with input and output devices such as keyboards, monitors, etc.

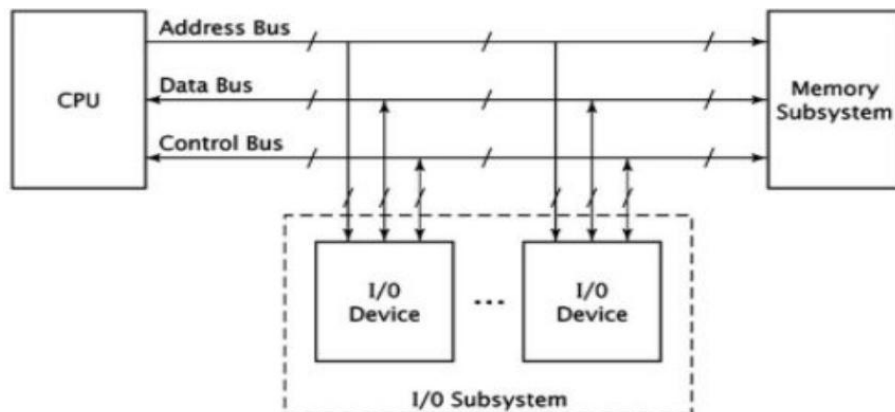


Fig: Basic Computer Organization

System buses

- Physically, a bus is a set of wires.
- A bus is a communication system that is used to transfer data between components inside a computer or between computers.
- As the complexity of a computer system increases, it becomes more efficient at using buses rather than direct connections between every pair of devices.
- The bus consists of three main parts. They are:

1. Address Bus

- When the CPU reads data or instructions from memory or writes data to memory, it must specify the address of the memory location it wishes to access. It outputs this address to the address bus.
- Simply, The CPU puts the address of the data it wants on the address bus, and memory fetches that data and sends it back to the CPU.
- It is unidirectional, meaning data flows from the CPU to other components.

2. Data Bus

- Data bus transfers data from one location to another across a computer.
- When the CPU needs to read or write data, it puts the data on the data bus, and it gets transferred between the CPU, memory, and other devices.
- It is bidirectional as the data flows in both directions between the CPU and memory.

3. Control Bus

- The control bus carries the control signal which is generated by the Control Unit (CU) of the CPU.
- The signal is used for controlling and coordinating various activities across the computer.
- It is bidirectional.
- Example: It can signal when to read data, when to write data and when an operation is complete. (Memory read or write, I/O read or write, Interrupt request, etc.)

(Asked 2 times)

2. Explain the basic CPU organization in detail with the help of block diagram.

- CPU performs various operations and controls the computer.
- It basically performs four operations, and they are fetch, decode, execute and store.
- Internally, the CPU has three sections. They are:
 - a. Register section
 - b. ALU (Arithmetic Logic Unit)
 - c. Control Unit

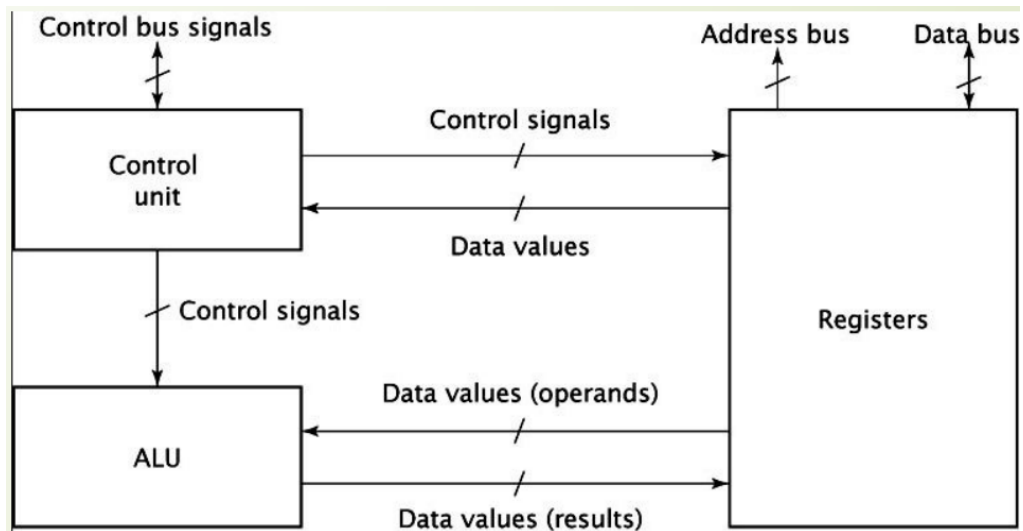


Fig: CPU Organization

a. Register Section

- The register section is like a storage area with special places to hold data within the CPU.
- It includes a set of registers, which are like small storage spots, and a communication pathway called a bus.
- Some registers are not directly accessible to programmers—they're like hidden compartments.
- These registers store temporary data that the CPU needs for its operations.

b. ALU (Arithmetic Logic Unit)

- The ALU is the part of the CPU responsible for doing arithmetical and logical operations.
- It obtains operands from the register section and stores the results back there.
- It's designed to work really fast, completing its calculations in just one clock cycle.

c. Control Unit (CU)

- Control Unit generates special signals that tell the registers to load data, increment data/numbers, clear contents, or output data.
- It also causes ALU to perform correct function.

(Asked 2 times)

3. Briefly explain the Instruction cycle in computer organization.

- The time taken for the execution of an instruction is known as the instruction cycle.
- It is the process by which a computer retrieves a program instruction from its memory, determines what actions the instruction describes, carries out those actions and store results if any produced.
- This cycle is repeated continuously by computer's CPU, from boot-up until the computer has shut down.
- An instruction involves 4 steps:

1. Fetch

- The CPU fetches the instruction from the memory.
- It sends the memory address of the instruction to the memory unit.
- The instructions are loaded in the Instruction Register (IR).
- PC (Program Counter) holds the address of the next instruction to be fetched and is incremented after each fetch.

2. Decode

- The control unit decodes (understands) the instruction in the IR.
- It figures out what operation the instruction is telling the CPU to perform.
- It also determines what data or operands are needed for the operation.

3. Execute

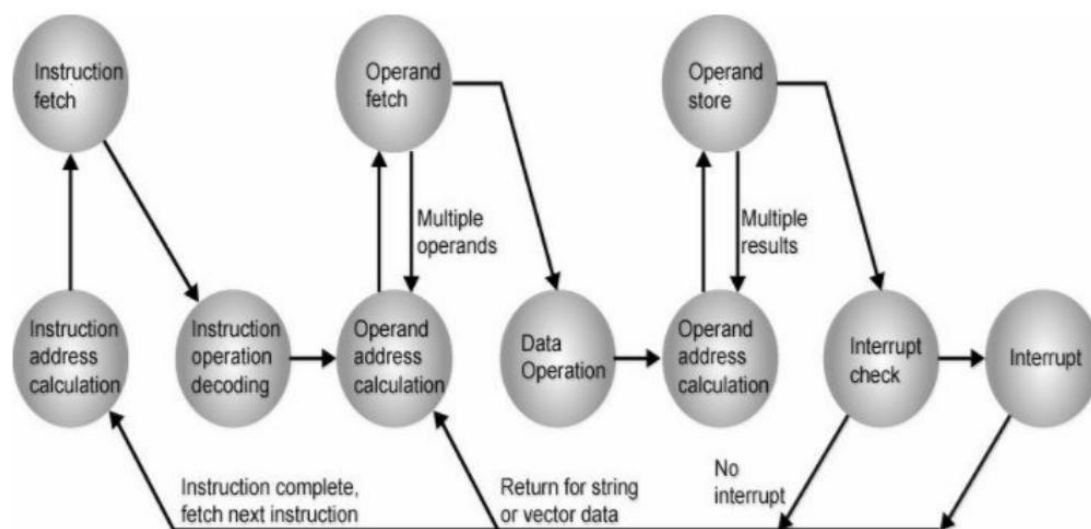
- The ALU (Arithmetic Logic Unit) performs the actual operation specified by the instruction.
- It uses data from registers or memory as operands.
- For example, it might add two numbers, compare values, or perform other calculations.

4. Store

- If the operation generates a result, the result is stored back in a register or memory.

(Not asked yet)

Q. Instruction cycle state diagram



Opcode: It is the operation to be performed by the CPU.

Operand: Data/Value on which operation is performed.

(Asked 2 times)

4. Difference between high order interleaving and low order interleaving.

➤ The differences are explained below:

1. Bit Selection:

- High-order interleaving focuses on arranging memory based on the most significant bits.
- Low-order interleaving focuses on arranging memory based on the least significant bits.

2. Conflicts:

- High-order interleaving aims to reduce conflicts for consecutive memory access by aligning addresses with similar high-order bits.
- Low-order interleaving aims to avoid conflicts when accessing memory locations that are near each other.

3. Purpose:

- High-order interleaving is chosen to optimize performance when accessing consecutive memory addresses sequentially.
- Low-order interleaving is chosen to minimize interference when accessing memory locations that are close to each other.

4. Access Patterns:

- High order interleaving benefits sequential memory access patterns, such as when processing arrays or lists.
- Low order interleaving is more effective when dealing with memory accesses that don't follow a strict pattern.

5. Trade-offs:

- High order interleaving might lead to conflicts for non-sequential memory access patterns.
- Low order interleaving might introduce inefficiencies when accessing consecutive memory locations.

Chapter 7

Memory Organization

(1 Q) (7 marks)

1. What is memory and its types.

- Computer memory is the storage space in a computer which is used to store data and instructions.
- There are two types of memory.
They are:

1. Primary memory

- This is the computer's quick memory.
- It's where programs run, and data is processed.
- They are volatile. This means that the data stored in these types of memory is temporary and gets erased when the computer is powered off or restarted.
- It includes registers, cache, and main memory (RAM).
 - a. Registers**
 - Super-fast memory inside the CPU for instant tasks.
 - b. Cache**
 - Cache memory is a small, high-speed memory located between the CPU and main memory.
 - c. RAM (Random Access Memory)**
 - It holds data the CPU is currently using.

2. Secondary Memory

- This is the computer's long-term storage.
- It is a non-volatile memory. It holds files and data even when the computer is off.
- It Includes hard drives, SSDs, Magnetic tapes, USB drive, and more.
 - a. Hard Drives / SSD**
 - Big storage for files, software, and data.
 - b. Magnetic Tape**
 - It is usually used for backing up large data.

(Asked 5 times)

2. What is memory hierarchy.

OR

Explain the memory hierarchy system with reference to cost, storage capacity, CPU access time, etc.

- The memory hierarchy is a system that arranges different types of computer memory in layers based on their speed, capacity, and cost.
- We have all the types of memory in a computer.
- Memory hierarchy allows computers to balance the need for fast access to data with the cost-effectiveness of storing large amounts of data.

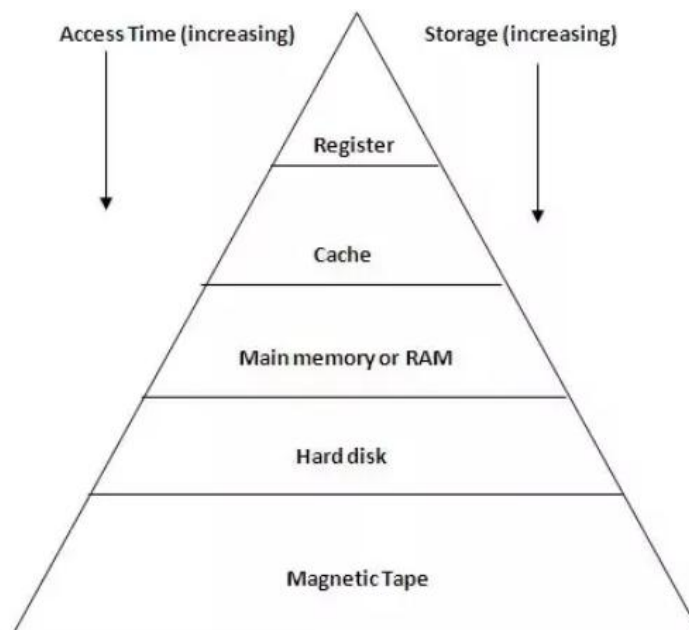


Fig: Memory hierarchy system

- Designing computer memory involves balancing three key factors: storage capacity, speed (or access time), and cost.
- However, these factors are interconnected, leading to trade-offs that influence the memory's efficiency.

Capacity:

- Capacity refers to how much data a memory component can store.

Speed / Access Time:

- Access time is the time it takes for the memory to provide data to the CPU.

Cost:

- Cost per bit is the amount of money required to store each bit of data.

Trade-Offs and Relationships:

- As access time decreases (faster access), the cost per bit increases. Achieving lightning-fast access usually involves more expensive technologies.
- Greater capacity often results in a smaller cost per bit. More storage for the same price.
- However, as capacity increases, access time tends to increase as well. Managing larger memory spaces requires additional time.

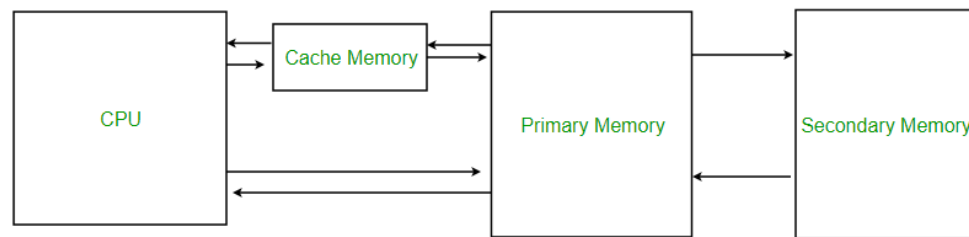
As we move down the memory hierarchy (top to bottom), we observe the following trends:

1. Capacity increases.
2. Cost per bit decreases.
3. Access time increases, meaning the data retrieval takes longer time.

(Asked 2 times)

3. What is Cache memory? What is the significance of cache memory.

- Cache memory is a type of high-speed memory that sits between the central processing unit (CPU) and the main memory (RAM) of a computer.
- It bridges the speed gap between the super-fast CPU and the slower main memory.
- Its primary purpose is to store frequently used data and instructions to provide quicker access to the CPU.
- Programs often access the same data or instructions repeatedly (temporal locality) and adjacent data (spatial locality).
- Cache memory takes advantage of this pattern by storing frequently used items.
- When the CPU requests data, cache memory checks if it's already stored. If yes, it's a cache hit, and the data is fetched quickly. If not, it's a cache miss, and the data is retrieved from main memory.



Significance

- Cache memory reduces the need for the CPU to wait for data from the main memory. This speeds up program execution, improving overall system performance.
- By storing frequently used data, cache memory optimizes CPU efficiency.
- Cache memory balances the speed of registers and the capacity of main memory.
- Cache memory reduces the need to access main memory repeatedly, improving overall system responsiveness.

4. Do you agree that the system works without cache? Give suitable reasons to support your answer.

- Yes, computer systems can indeed work without cache memory, but the absence of cache memory would lead to noticeable performance drawbacks.
- They are:
 - a. Slower Data Access:**
 - Without cache, the CPU would need to fetch data from slower main memory every time it's needed.
 - This results in longer wait times for data, slowing down program execution.
 - b. Higher Memory Traffic:**
 - Cache reduces the need for constant main memory access.
 - Without cache, each data request increases memory traffic, potentially causing congestion.
 - c. Processor Idle Time:**
 - Lack of cache might lead to CPU waiting for data, causing processor idle time.
 - This can reduce overall system efficiency.
 - d. Limited Real-time Performance:**
 - Real-time applications rely on fast data access for responsiveness.
 - Cache absence could introduce delay.

(Asked 2 times)

5. What is associative memory? How does it work?

- Associative memory, also known as content addressable memory (CAM), is a type of memory that allows you to search for data based on its content, rather than using specific memory addresses.
- Traditional memory needs an address to retrieve data.
- Associative memory lets you search using the actual content of the data.

Working Mechanism:

- Data is stored alongside a "tag" that represents the content.
- When you search, the memory compares your search content with the stored tags.
- If it finds a match, it provides the corresponding data.

- Associative memory performs searches in parallel, checking all stored data simultaneously.
- This makes searches incredibly fast compared to traditional memory.
- Associative memory is handy for tasks like database searches, finding duplicates, or matching patterns in data.
- This memory is designed for quick lookups, which makes it great for real-time applications.
- Example: Searching for a student's information by using his first name only in the database.

(Asked 3 times)

6. Explain different types of cache mapping techniques.

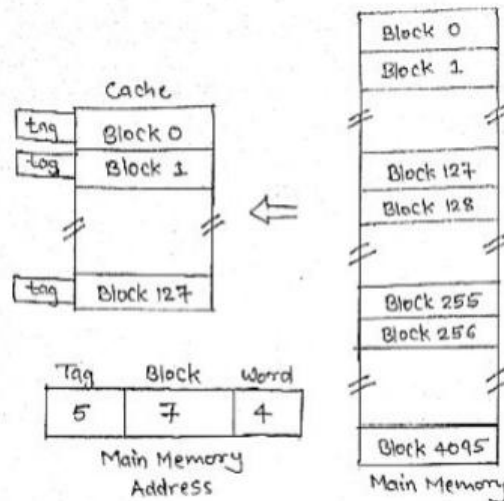
OR

Briefly explain direct mapping, associative mapping and set associative mapping.

- The primary purpose of cache is to store frequently used data and instructions to provide quicker access to the CPU.
- Mapping, in the context of cache memory, refers to the process of determining where blocks of data from the main memory are placed within the cache.
- Data from main memory is stored in specific locations within the cache.
- This speeds up data access because the computer knows exactly where to look.
- Mapping techniques determine how data is assigned to these locations, helping the computer quickly find and use important data.
- There are three types of mapping techniques.
 - a. Direct mapping
 - b. Set-associative mapping
 - c. Associative mapping / Fully associative mapping

a. Direct Mapping:

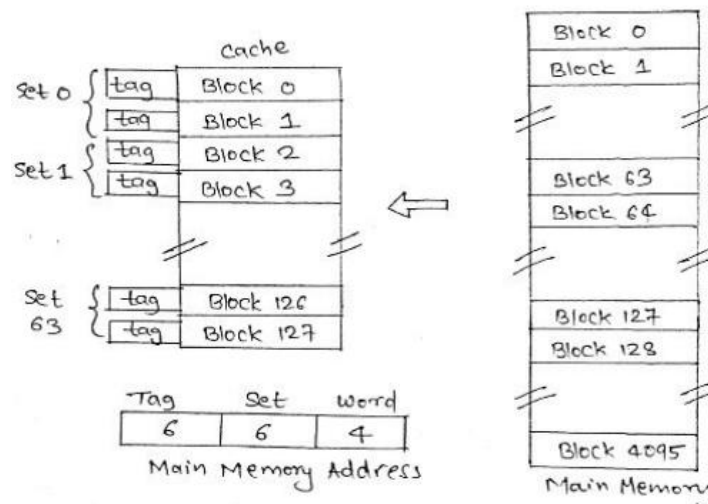
- Each data block from the main memory is stored in a specific cache location (cache line).
- When the CPU wants data, it checks the specific cache line.
- It is easy to implement but can lead to conflicts if multiple blocks want the same cache location.



Here, the size of tag is 5-bits, block is 7 bits and word is 4 bits.

b. Set-Associative Mapping:

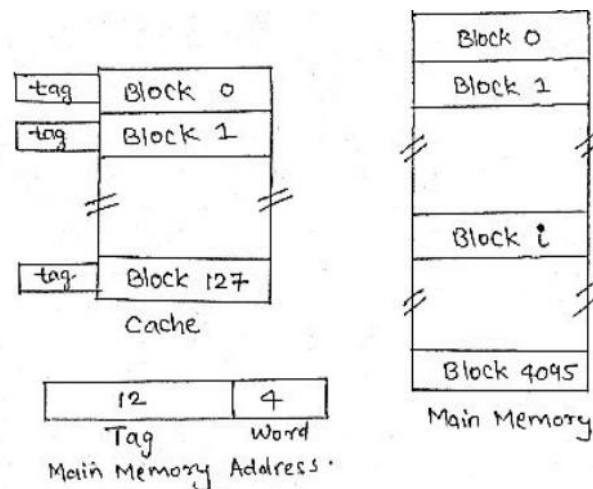
- Cache is divided into sets, each containing multiple cache lines.
- Data blocks from main memory can be placed in any cache line within a set.
- When the CPU wants data, it checks within the set.
- It reduces conflicts compared to direct mapping.
- It balances performance and complexity.



Here, the size of tag is 6-bits, set is 6 bits and word is 4 bits.

c. Fully Associative Mapping (or Associative mapping):

- In this approach, each data block from main memory can be placed anywhere in the cache, without being restricted to specific locations.
- There are no restrictions on where a block is stored, it is very flexible.
- There are no conflicts.
- It requires complex hardware for efficient tag matching.
- However, due to the complexity of searching through all cache lines simultaneously, it can be slower than other methods.



Here, the size of tag is 12-bits and word is 4 bits.

7. Describe the four most commonly used replacement algorithms related to design issues of cache memory.

- Replacement algorithms are needed in cache memory systems to efficiently manage the limited space available in the cache.
- The main purpose of replacement algorithms is to decide which cache lines to evict or replace when new data needs to be loaded into the cache.
- The most commonly used replacement algorithms are:

a. Random Replacement:

- This algorithm replaces a randomly chosen cache line when a new data block needs to be loaded.

b. FIFO (First-In-First-Out):

- In FIFO, the cache line that has been in the cache the longest is replaced when a new block is loaded.

c. LRU (Least Recently Used):

- LRU replaces the cache line that hasn't been used for the longest time when new data is loaded.

d. LFU (Least Frequently Used):

- LFU replaces the cache line that has been used the least number of times.

8. What are the strategies to write data to cache memory.

- There are generally two common strategies used to write data to cache memory: Write-Through and Write-Back.

a. Write-Through:

- In this strategy, when a CPU writes data to cache, the data is also immediately written to the main memory.
- This ensures that the data in cache and main memory are always consistent.
- While it maintains data integrity, it can slow down write operations since both cache and main memory need to be updated.

b. Write-Back:

- With this strategy, when the CPU writes data to cache, it updates the cache but doesn't immediately update the main memory.
- The data is marked as "dirty" or modified in the cache.
- When the cache line is replaced, then only the modified data is written back to the main memory.
- Write-back is more efficient because it optimizes performance for write-heavy workloads as it reduces the frequency of main memory writes.

9. What is cache performance?

- Cache performance refers to how well cache helps speed up a computer.
- It's about making sure the cache stores the right data that the computer needs, so the computer can quickly get the data it wants without waiting for slower memory.
- Good cache performance means the computer runs faster and smoother.

Cache performance is evaluated based on metrics such as hit ratio, miss ratio, average access time, etc.

- Every time the CPU accesses memory, it checks the cache. If the requested data is in the cache, it accesses data from cache rather than physical memory, this is a **cache hit**.
- If the data is not in the cache, the CPU accesses the data from main memory (and usually writes the data into the cache as well). This is a **cache miss**.
- **Hit ratio** is the ratio of total number of hits to the total number of CPU references.
- **Average access time** is the time taken to find whether the required element is in cache or not.

(Asked 5 times)

10. Explain Virtual memory. Why do we need it?

- Virtual memory is a technique which allows programs to use more memory than it is physically available in the main memory (RAM).
- It allows programs to use more memory than what is physically installed in the computer.
- In virtual memory, parts of a program that are not currently needed are temporarily moved from main memory to secondary storage such as hard disk.
- This frees up main memory for other critical tasks.
- When the swapped-out parts are needed again, they can be swapped back into memory.
- When a process is swapped out, its state is saved to the disk, and when it is swapped back in, its state is restored.
- Swapping can be done manually or automatically by the operating system.
- It can be useful in real-time systems to ensure that critical tasks have enough memory to execute.
- Virtual memory helps prevent running out of memory and allows larger programs to run.
- However, swapping can introduce additional overhead and can cause performance degradation.

(Asked 6 times)

11. Differentiate between Paging and Segmentation.

- Paging and segmentation are memory management techniques used within virtual memory systems, and they are closely related to the concept of memory swapping.
- Memory swapping involves moving data between the computer's main physical memory (RAM) and secondary storage (like a hard disk) to efficiently manage memory usage.
- Paging and segmentation provide ways to organize and manage the memory space, making it easier to move data in and out of physical memory when needed.

a. Paging

- Paging is a memory management technique in the operating system.
- Paging is a technique where the program's logical address space (secondary storage) is divided into fixed-size blocks called pages, and physical memory (main memory) is divided into corresponding blocks called frames.
- In paging both main memory and secondary memory is divided into fixed size partitions.
- The size of pages and frame is kept the same for optimum utilization of main memory and to avoid external fragmentation.
- The operating system uses a page table to map each page's logical address to the corresponding physical address.
- When a program is executed, its memory is divided into pages, and these pages can be scattered anywhere in physical memory.
- Paging helps manage memory allocation more efficiently and reduces external fragmentation.

b. Segmentation:

- Segmentation divides the program's logical address space into variable-sized segments, each representing a different part of the program, like code, data, stack, etc.
- The segmentation method works almost similarly to paging, the only difference between the two is that segments are of variable length whereas, in the paging method, pages are always of fixed size.
- Information related to each segment is stored in a table known as segment table.
- The segment table maps the program's segments to their corresponding physical memory locations.
- Segment tables generally occupies less space as compared to paging tables.
- Segmentation provides a more structured way to manage memory, making it easier to handle different parts of a program separately.
- However, it can lead to internal fragmentation if segments are of varying sizes.
- In a virtual memory system, segmentation can work alongside paging to further optimize memory usage.

(Differentiation is asked most of the time so you can directly remember this table)

S.N	Paging	Segmentation
1	Memory is divided into fixed-sizes blocks known as pages and frames.	Memory is divided into variable-sized blocks known as segments.
2	For paging OS is accountable.	For segmentation compiler is accountable.
3	Page size is decided by the hardware.	Segment size is specified by the user.
4	Paging is not visible to the user.	Segmentation is visible to the user.
5	This technique can lead to internal fragmentation	This technique can lead to external fragmentation.
6	Uses page table to map pages to frames.	Uses segment table to map segments to memory.
7	Paging table occupies more space than segmentation table.	Segmentation table occupies less space than paging table.
8	The size of the page should always be equal to the size of frames.	There is no constraint on the size of segments.
9	In paging, programmer cannot efficiently handle data structure.	It can efficiently handle data structure.
10	Paging results in a less efficient system.	Segmentation results in a more efficient system.

Q. "Paging as one-dimensional virtual memory management".

Ans: Explain paging.

Q. How is page table used to convert logical address to physical address.

(Asked 2 times)

12. Explain the importance of cache memory and virtual memory.

Cache memory

- Cache memory reduces the need for the CPU to wait for data from the main memory. This speeds up program execution, improving overall system performance.
- By storing frequently used data, cache memory optimizes CPU efficiency.
- Cache memory balances the speed of registers and the capacity of main memory.
- Cache memory reduces the need to access main memory repeatedly, improving overall system responsiveness.

Virtual memory

- It allows programs to use more memory than what is physically installed in the computer.
- This frees up main memory for other critical tasks.
- It's essential for running large applications smoothly, preventing memory shortages and crashes.
- Each program gets its own virtual space, ensuring they don't interfere with each other and enhancing system security.
- It can be useful in real-time systems to ensure that critical tasks have enough memory to execute.
- Virtual memory helps prevent running out of memory and allows larger programs to run.

Q. Differentiate cache memory and virtual memory.

Chapter 8

Input/Output Organization

(1 or 2 Que) (7-15 marks)

1. What is synchronous and asynchronous data transfer.

- **Synchronous data transfer** is a method where devices share data at fixed, regular intervals, often guided by a common clock signal.
- This ensures that both sending and receiving devices work in sync, coordinating their actions to exchange information smoothly and predictably.
- Synchronous data transfer is highly effective when the peripherals.
- This method is often used in scenarios where precise timing and coordination are essential, such as voice and video streaming.

- **Asynchronous data transfer** refers to a method of data transmission where data is sent and received without requiring the sender and receiver to be synchronized in real-time.
- The sender transmits data without waiting for an immediate response from the receiver.
- The receiver processes the data whenever it becomes available, without needing to be synchronized with the sender's clock.
- This method is commonly used for various types of computer communication, such as email, file transfer, and web browsing.

Q. What is handshaking?

- Handshaking refers to the exchange of control signals between devices to ensure coordinated and synchronized data transfer.
- Simply, it is a way of making sure both sides are ready to share information.
- It involves signals like "Ready" and "Valid" to indicate when a device is prepared to send or receive data.
- Handshaking prevents data loss or corruption by ensuring that data is transferred only when both devices are ready.

(Asked 2 times)

2. Explain different modes of asynchronous data transfer.

OR

How is asynchronous data transfer done in CPU.

- The four types of asynchronous data transfer modes are:
 - a. Source-Initiated Data Transfer:**
 - In this mode, the source device initiates the data transfer when it has data to send. Example: Sending email.
 - The destination device is assumed to be ready to receive the data.
 - No handshaking signals are involved, so the source sends the data whenever it's ready.
 - b. Destination-Initiated Data Transfer:**
 - In this mode, the destination device triggers the data transfer when it's ready to receive data.
 - The source device is assumed to have data available for transmission.
 - Like source-initiated, no handshaking signals are used; the destination indicates its readiness.
 - Example: Flash drive being ready to receive data.
 - c. Source-Initiated Data Transfer with Handshaking:**
 - This mode incorporates handshaking signals to ensure synchronized communication.
 - The source asserts a "Ready" signal when it has data, and the destination responds with a "Valid" signal when it's ready to accept data.
 - Data transfer occurs only when both source and destination signals align, ensuring reliable exchange.
 - d. Destination-Initiated Data Transfer with Handshaking:**
 - Similar to the previous mode, but initiated by the destination.
 - The destination asserts a "Ready" signal, prompting the source to send data.
 - Once the source sends data, it asserts a "Valid" signal, and the destination acknowledges with a "Done" signal.

Q. What is I/O Interface.

- The method that is used to transfer information between internal storage and external I/O devices like printers, keyboard, etc is known as I/O interface.

Mode of Transfer:

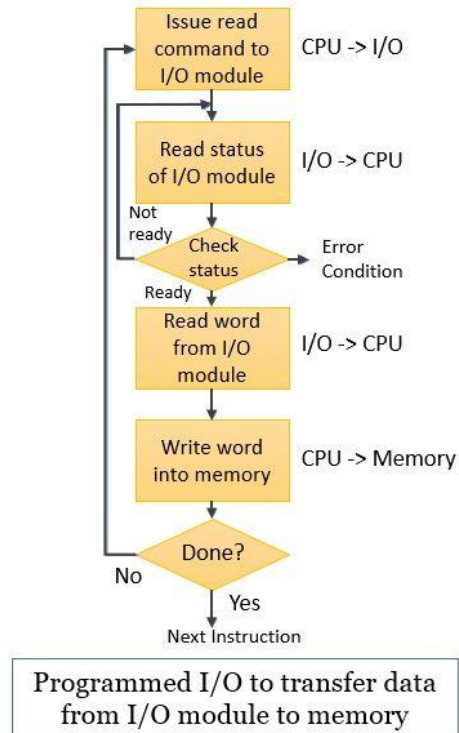
- Data transfer to and from the peripherals (printer, keyboard, monitor) may be done in any of the three possible ways:
 - a. Programmed I/O.
 - b. Interrupt- initiated I/O.
 - c. Direct memory access (DMA).

(Asked 3 times)

3. Explain Programmed I/O.

- Programmed I/O is a method where the central processing unit (CPU) takes full charge of transferring data between itself and external devices, such as printers or keyboards.
- In this approach, the CPU gives explicit commands to these devices to send or receive data.
- To ensure effective communication, the CPU regularly checks the status of these devices to see if they're ready to exchange data.
- This involves a continuous loop where the CPU repeatedly asks, "Are you ready yet?"
- However, this constant monitoring by the CPU is resource-intensive and not very efficient.
- It keeps the CPU occupied even when there might not be any data transfer happening.

- A more efficient approach is using interrupts. Instead of the CPU constantly asking, devices themselves send a special signal, called an interrupt request, to the CPU when they're ready for data transfer.
- This way, the CPU can focus on its main tasks until it's genuinely needed for data exchange.



(Asked 2 times)

4. What is an interrupt? Explain different types of interrupt.

- An interrupt is a signal sent by a hardware device or a software program to the central processing unit (CPU) to gain its attention.
- Interrupts indicate that an event requiring immediate processing has occurred. Or when device is ready to transfer data, it sends interrupt request signal to the CPU.
- They allow the CPU to temporarily pause its current task, handle the event, and then resume its previous activity.

Types of Interrupts:

a. Internal Interrupts:

- These are generated by the CPU itself when it detects some exceptional condition during the execution of a program.
- Examples include division by zero or invalid instruction.

b. External Interrupts:

- These are generated by external hardware devices to request attention from the CPU.
- Generally used by CPU to interact with I/O device.
- Common sources of external interrupts are keyboard presses, mouse clicks, or hardware malfunctions.

c. Software Interrupts:

- Also known as system calls, these are initiated by software programs to request services from the operating system.
- For example, when a program needs to read a file or allocate memory, it triggers a software interrupt to get the OS's help.

(Asked 3 times)

5. Interrupt driven I/O.

OR

What is an interrupt handling. Write typical steps in interrupt handling.

How is interrupt serviced? Explain software actions in interrupt handling.

- Interrupt-driven I/O is a technique where devices initiate interrupts to inform the CPU that they are ready for data transfer.
- This technique overcomes the limitation of Programmed I/O.
- Instead of the CPU constantly checking device status, it can focus on other tasks until the device signals that it's ready.
- When the CPU detects an interrupt signal, the CPU saves the state and jumps to the ISR(Interrupt Service Routine).
- The interrupt handler determines the cause of interrupt and performs the necessary processing.
- The CPU returns to the execution state prior to the interrupt.

Interrupt Handling:

- Interrupt handling involves managing the process when an interrupt occurs.
- Steps in interrupt handling typically include:
 1. **Interrupt Detection:** The CPU recognizes the interrupt request from the device.
 2. **Interrupt Acknowledgment:** The CPU acknowledges the interrupt and saves the current state of the program.
 3. **Interrupt Servicing:** The CPU switches to a specific ISR (Interrupt Service Routine) to handle the interrupt's task. ISR fulfills the purpose of the interrupt.
 4. **Interrupt Return:** After completing the task, the CPU restores the previous state and resumes the original program from where it was interrupted.
- An interrupt causes CPU to transfer the control temporarily from its current program to another program i.e. interrupt handler.

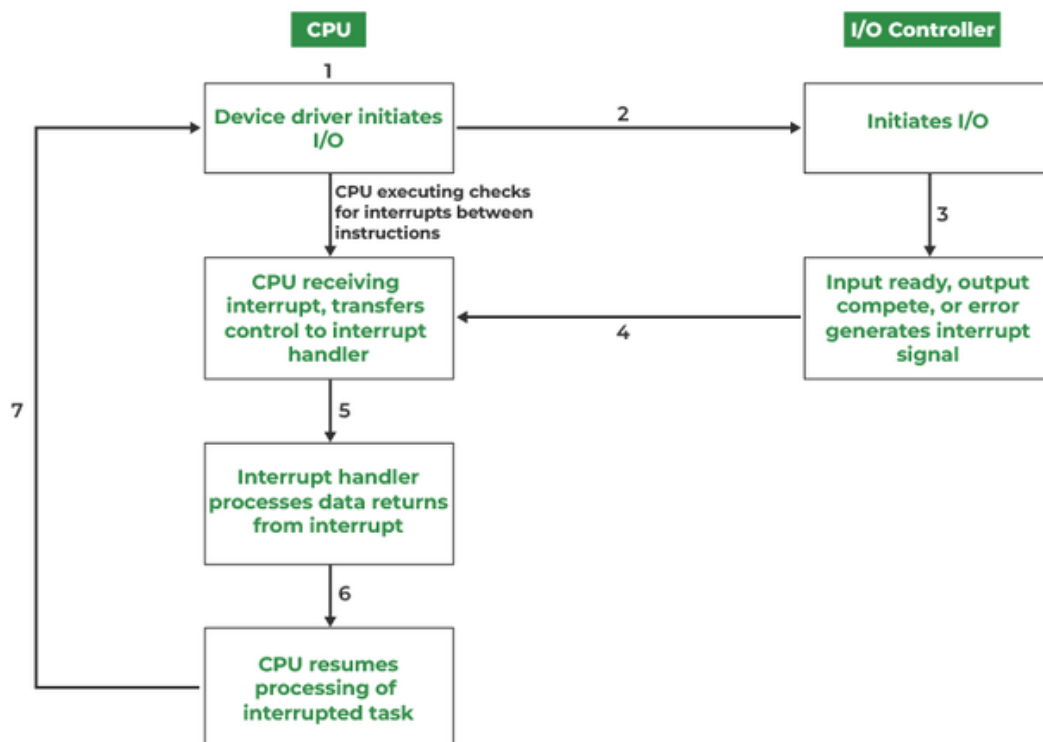


Fig: Interrupt Driven I/O Cycle

(Asked 2 times)

6. Explain Interrupt Vector.

- An interrupt vector is a table of addresses that holds the memory addresses of the interrupt service routines corresponding to different types of interrupts.
- When an interrupt occurs, the CPU uses the interrupt vector to locate the appropriate ISR and execute it.
- It's like having a map that directs you to the right place when you encounter different situations.

Q. Programmed Driven I/O Vs Interrupt Driven I/O.

(Asked 11 times) VVIP

7. Describe DMA

OR

What is DMA? Describe how it works with a proper diagram.

OR

How DMA controller can be incorporated in a computer system.

- DMA stands for Direct Memory Access.
- It is a technique that allows data transfer within main memory and external devices without involving the CPU for every data transfer.
- It significantly speeds up data transfer and reduces CPU involvement, making it ideal for large data movements.
- The CPU can perform other important tasks.
- This technique overcomes the drawbacks of the other two I/O techniques which were time consuming.
- It is more efficient to use DMA when a large volume of data has to be transferred.

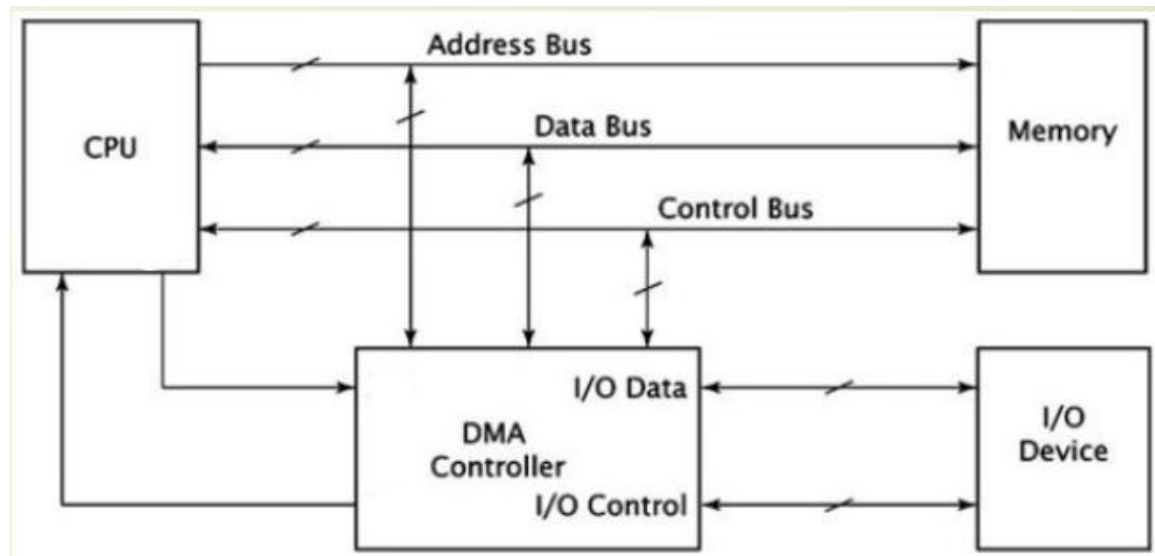


Fig: DMA Controller for Data Transfer

Working of a DMA

- When a peripheral/external device wants to transfer data, it sends a request to the DMA controller.
 - The CPU grants permission to the DMA controller, specifying the source, destination, and number of data bytes to be transferred.
 - The DMA controller takes control of the system bus and initiates the data transfer between the peripheral and memory without CPU intervention.
 - Once the transfer is complete, the DMA controller interrupts the CPU, which then processes the transferred data.
-
- A DMA controller can be integrated into a computer system by connecting it to the system bus.
 - It communicates with the CPU, peripherals, and memory using control lines and data lines.
 - The CPU initializes the DMA controller, specifies transfer details, and grants access to system resources.

(Asked 2 times)

8. Explain different DMA transfer modes.

➤ DMA Transfer Modes are explained below:

1. Burst Mode:

- Entire block of data is transferred in one contiguous sequence.
- Once the DMA controller is granted access to the system buses by the CPU, it transfers all bytes of data in the data block before releasing control of system bus back to CPU.

2. Cycle Stealing Mode:

- The DMA controller and the CPU take turns using the system bus.
- It overcomes the limitation of burst mode where CPU has to wait until DMA completes its work.
- DMA steals cycles when CPU isn't actively using the bus.
- Cycle stealing mode is useful when the CPU and DMA need to operate concurrently without significantly impacting each other's performance.

3. Transparent Mode

- DMA controller only transfers data when the CPU is performing operations that do not make use of system bus.
- It's designed to operate in the background without requiring CPU intervention.
- In this mode, the DMA controller monitors the bus and initiates transfers when it detects bus inactivity.
- Transparent mode is efficient for transferring small data without causing disruption to CPU tasks.
- The primary advantage is that CPU never stops executing its programs.

9. What are I/O processors. Explain how they work using suitable diagrams.

- I/O processors, are specialized components that manage input and output operations between a computer's main processing unit (CPU) and various peripheral devices.
- Their main purpose is to offload the CPU from the task of handling the details of data transfer and communication with external devices, allowing the CPU to focus on more critical processing tasks.
- When a peripheral device (such as a disk drive or a printer) needs to communicate with the CPU, it sends a signal to the I/O processor to initiate a data transfer.
- The I/O processor acts as a mediator between the peripheral and the CPU.
- The I/O processor handles the actual transfer of data between the peripheral and the computer's memory.
- Once the data transfer is complete, the I/O processor signals the CPU through an interrupt.
- The CPU then interrupts its ongoing tasks and checks the I/O processor.
- The CPU interacts with the I/O processor to collect the data or send data for processing.

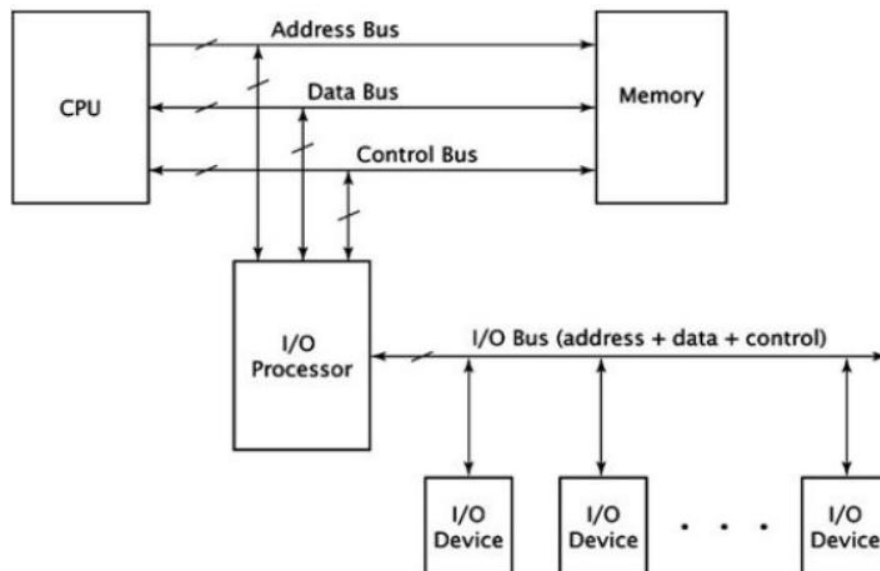


Fig: I/O Processor

10. What is Serial Communication. Explain UART, RS 232 and USB.

- Serial communication is a method of transferring data between devices one bit at a time sequentially over a single communication line.
- It contrasts with parallel communication, where multiple bits are transferred simultaneously on separate lines.
- Serial communication is commonly used when simplicity, longer distances, or limited available communication lines are factors.

UART (Universal Asynchronous Receiver-Transmitter):

- UART is a hardware module that facilitates serial communication by converting parallel data from a computer's memory into serial data for transmission and vice versa.
- It enables asynchronous communication, where data is sent without a clock signal, using start and stop bits to delineate each data packet.
- UART is often integrated into microcontrollers, embedded systems, and communication interfaces to enable serial communication between devices.

RS-232 (Recommended Standard 232):

- RS-232 is a standard defining the electrical characteristics and communication protocols for serial communication between devices, often used for connecting computers and peripherals.
- It specifies signal voltage levels, data format, and connector pin assignments.

USB (Universal Serial Bus):

- USB is a widely used standard for connecting devices to computers and other electronic devices.
- It allows multiple devices to connect through a single interface using a variety of connectors, such as USB-A, USB-B, and USB-C.
- USB supports higher data rates than RS-232 and provides features like hot-swapping (connecting or disconnecting devices without powering down), etc.

- USB is used for a wide range of peripherals, including printers, storage devices, cameras, and more.

In summary, serial communication involves transmitting data sequentially over a single line, and UART serves as a hardware module to enable this communication. RS-232 is a standard defining serial communication parameters, and USB is a versatile standard for connecting various devices to computers using different connectors and higher data rates.

Chapter 9

Introduction to RISC

(1 Q) (7 marks)

1. Explain major set of design principle related to RISC principle.

- RISC (Reduced Instruction Set Computer) design principles focus on making computer processors simpler and faster by using a smaller set of simple and fast instructions.
- Here are the major design principles of RISC in simple words:
 1. **Simplicity:** RISC processors use a small and straightforward set of instructions that can be executed quickly. Each instruction performs a specific task, making the processor easier to design and understand.
 2. **Fast Execution:** RISC instructions are designed to be executed in a single clock cycle, making the processor faster. This efficiency helps complete tasks quickly and improves overall system performance.
 3. **Fixed-Length Instructions:** RISC instructions have a fixed length, which allows the processor to fetch and decode instructions more efficiently. This leads to faster execution and better predictability.
 4. **Pipeline:** RISC processors use a pipeline to break down instruction execution into stages. This allows different instructions to be processed simultaneously, boosting efficiency and speed.
 5. **Single-Cycle Execution:** Each instruction is designed to complete in a single clock cycle. This streamlined approach minimizes delays and optimizes overall processing speed.
 6. **Less Complex Instructions:** RISC instructions perform simple tasks, reducing the complexity of each instruction. This simplicity improves the predictability of execution and helps avoid unexpected behaviors.

(Asked 4 times)

2. RISC Vs CISC

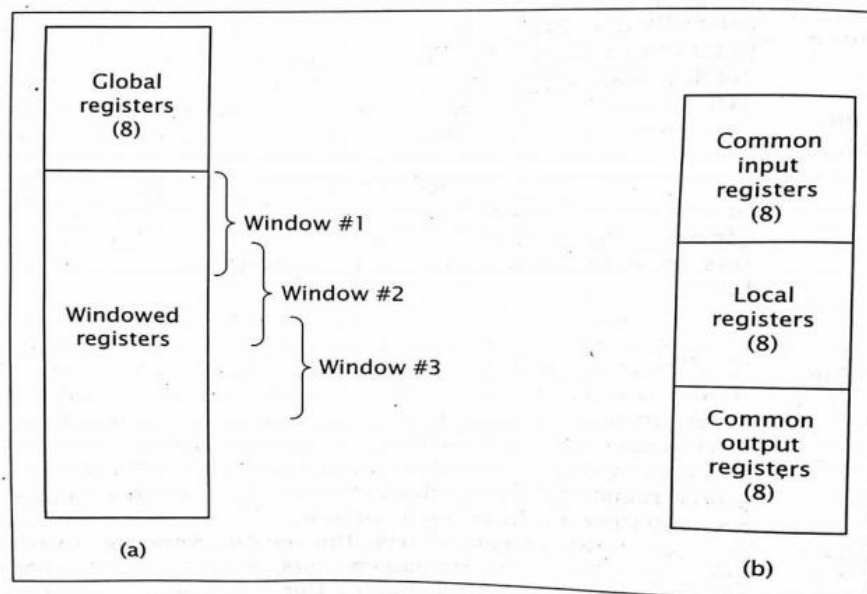
S. N	RISC	CISC
1	It stands for Reduced Instruction Set Computer.	It stands for Complex Instruction Set Architecture.
2	It has less number of instructions.	It has large number of instructions.
3	It has few numbers of addressing modes.	It has large number of addressing modes.
4	It has fixed length instruction format	It has variable length instruction format.
5	It requires single cycle for execution.	It requires more than one cycle for execution.
6	The cost of RISC is less.	The cost is high.
7	It is less powerful.	It is more powerful.
8	Execution time is very high.	Execution time is very less.
9	Example: VAX, Pentium Pro, etc.	Example: UltraSPARC, Power PC, etc.

(Asked 8 times)

3. Explain Register Window.

- A register window is like a special space in a computer's memory that holds a set of registers.
- These registers are like small storage areas that the processor uses to quickly work with data.
- Simply, the register window has a group of registers that can hold data and instructions while the processor is working on tasks.
- The register window isn't infinite; it has a specific number of registers. This limit is designed to keep things organized and efficient.
- When the processor switches to a different task, it might change the registers in the window. The old data is saved, and new data is brought in for the new task.
- This helps the processor work more efficiently by having the right registers ready for the current task.
- Register windows are often found in processors that follow RISC design.
- The CPU can access data in register more quickly than data in memory.
- Although a RISC processor has many register, it may not be able to access all of them at any time.
- Most RISC CPU has some global register which are always accessible.

Register windowing in the SPARC processor



Date: _____

Page: _____

Chapter 9 Numerical

11111

- 1) In a system there are 5 windows of registers, each register share 4 input registers and 4 output registers. Each of the window has 10 ~~total~~ local registers. The system has 20 global registers. Calculate the total number of registers in the system. show the pictorial representation as well.

Soln:- here,

Total number of windows = 5

Number of ~~shared~~ ^{shared} input/output registers = 4
(shared/common input/output registers)

Total number of local registers in a window = 10

Total number of global registers in the system = 20

Now,

Total number of registers = Total number of global registers +
number of windows $\times$ (common input/output registers + no. of local registers)

$$= 20 + 5(4 + 10)$$

$$= \cancel{70} 20 + 70$$

$$= 90 \text{ registers}$$

Date: _____

Page: _____

Pictorial representation as stack

Global registers (20)	
Common input registers (4)	} window #1
local registers (10)	
Common output registers (4)	} window #2
local registers (10)	
Common output registers (4)	} window #3
local registers (10)	
Common output registers (4)	} window #4
local registers (10)	
Common output registers (4)	} window #5
local output registers (10)	
Common output registers (4)	

Here, the common ~~input~~ output registers of each window also act as common input registers for the next window.

Formula :

Let, ~~window~~ No. of windows (W)

No. of shared input/output registers (C)

No. of local registers (L)

No. of global registers (G)

Total number of registers = $G + W(L + C)$

Window Size = $L + 2C + G$

Date: _____

Page: _____

2) Calculate the window size and total number of registers. Given Number of global registers = 10, No. of local registers = 10, No. of common registers = 6, No of window = 4

soln: Here,

$$W = 4$$

$$C = 6$$

$$L = 10$$

$$G = 10$$

Now,

$$\begin{aligned}\text{Window Size} &= L + 2C + G \\ &= 10 + 12 + 10 \\ &= 32\end{aligned}$$

$$\begin{aligned}\text{Total number of registers} &= G + W(L + C) \\ &= 10 + 4(10 + 6) \\ &= 74\end{aligned}$$

4. Show how instruction pipelining can improve the performance of a system.

- An instruction pipeline is a fundamental concept in computer architecture that improves the efficiency of instruction execution in a processor.
- It's a technique used to overlap and parallelize the various stages of instruction processing, allowing multiple instructions to be processed simultaneously.
- This results in faster overall execution and better utilization of the processor's resources.
- Here's how it works:

Fetch Stage: The pipeline starts by fetching the next instruction from memory.

Decode Stage: The fetched instruction is decoded to determine the operation to be performed and the operands involved.

Execute Stage: The operation is performed, which might involve arithmetic calculations, logical operations, or memory accesses.

Memory Stage: If necessary, the instruction interacts with memory, such as loading or storing data.

Write Back Stage: The result of the operation is written back to the appropriate register or memory location.

- In an instruction pipeline, each stage is like a station on an assembly line.
- While one instruction is in the decode stage, the next instruction can be in the fetch stage, and so on.
- This allows multiple instructions to be in different stages simultaneously, increasing throughput and overall performance.

Instruction pipelines are common in modern processors, including both RISC and CISC architectures, as they significantly enhance the efficiency of instruction execution by allowing multiple instructions to be processed concurrently.

(Asked 6 times)

5. Explain different instruction pipelining conflicts (hazards).

OR

What are the major conflicts occurring due to instruction pipelining in RISC?

➤ The different types of conflicts that occur in instruction pipelining are:

1. Data conflict
2. Branch conflict

1. Data Conflict

- In an instruction pipeline, tasks are broken down into stages, different stages are working on different parts of different tasks at the same time.
- This allows multiple instructions to be in different stages simultaneously, improving the overall performance.
- However, when instructions depend on the results of previous instructions, data conflicts can occur.
- Consider the following consecutive program statements:
1: $R1 \leftarrow R2 + R3$
2: $R4 \leftarrow R1 + R3$
3: $R5 \leftarrow R6 + R3$
- In the first instruction, we are adding the values of R2 and R3 and storing the result in R1.
- The second instruction attempts to add the value of R1 (from the previous instruction) and R3, and then store the result in R4.
- If the second instruction proceeds before R1 is updated in the memory, it will use the old value of R1.
- This can lead to incorrect results.

2. Branch Conflict

- Think of branches in the pipeline like road intersections.
- When the processor encounters a branch instruction (a decision point), it needs to decide which path to take.
- The pipeline needs to decide whether to keep fetching and decoding instructions along the original path or the new path determined by the branch.
- This creates a challenge because the pipeline has to wait until it knows the outcome of the branch before it can proceed, which can slow down the execution.
- So, a branch conflict happens when the pipeline encounters a branch instruction.

The following code segment illustrates the branch conflict problem:

1: $R1 \leftarrow R2 + R3$

2: $R4 \leftarrow R5 + R6$

3: JUMP 10

4: $R7 \leftarrow R8 + R9$

5: $R10 \leftarrow R11 + R12$

:

10: $R13 \leftarrow R14 + R15$

After instruction 3 is executed, the CPU should branch to instruction 10; however, instructions 4 and 5 are already in the pipeline before instruction 3 is executed.

This can lead to wasted processing and time when the program takes a different branch than expected.

(Asked 6 times)

6. What are the solutions to correct conflicts that occur in instruction pipelining.

OR Describe how data and branch conflict can be resolved.

Data Conflict Solutions:

1. No-Op (No Operation):

- The simplest way is to have the compiler detect data conflicts and insert no-ops to avoid the conflict.
1: $R1 \leftarrow R2 + R3$
N: no-op
2: $R4 \leftarrow R1 + R3$
3: $R5 \leftarrow R6 + R3$
- The no-op statement in the second block delays the fetching of operands for the second instruction by one clock cycle.
- This delay allows the last stage of the pipeline to store the new value of R1 before it is loaded for use in executing the second instruction, thus avoiding data conflict.

2. Instruction Reordering:

- The compiler or hardware can rearrange the order of instructions to minimize data conflicts.
- Instructions that don't depend on the result of a conflicting instruction are moved ahead.
1: $R1 \leftarrow R2 + R3$
3: $R5 \leftarrow R6 + R3$
2: $R4 \leftarrow R1 + R$

3. Waiting or Stall:

- Additional hardware is added to RISC pipeline.
- The hardware detects data conflicts between instructions in the pipeline and inserts stalls, or introduces delays, to resolve the data conflicts.
- It is same like no-op insertion, except it is handled by hardware.

Branch Conflict Solutions:

1. No-Op (No Operation):

- When a branch instruction is in conflict, a no-op (instruction with no effect) is inserted into the pipeline.

Example:

1: $R1 \leftarrow R2 + R3$

2: $R4 \leftarrow R5 + R6$

3: JUMP 10

N1: no-op

N2: no-op

4: $R7 \leftarrow R8 + R9$

5: $R10 \leftarrow R11 + R12$

:

10: $R13 \leftarrow R14 + R15$

- The no-ops introduce a delay sufficient to ensure that instructions 4 and 5 are never introduced into the pipeline.

2. Instruction Reordering:

- Similar to data conflicts, instructions can be reordered to minimize the impact of branch conflicts.
- Instructions not dependent on the branch outcome are moved ahead.

3. Annul or Flush:

- If a branch instruction is in conflict, the pipeline can be flushed, meaning all instructions after the branch are invalidated.
- This clears the pipeline of any incorrect instructions.

4. Branch Prediction:

- Hardware can predict whether a branch will be taken or not and start executing instructions accordingly.

Chapter 10

Introduction to Parallel Processing

(1Q) (7 marks)

1. Differentiate between Serial and Parallel Communication.

- Serial and parallel communication are two different methods used to transfer data between devices or components.
- They differ in how data is transmitted.

Serial Communication

- In serial communication, data is sent one bit at a time over a single communication channel or wire
- Serial communication requires fewer wires since data is sent sequentially.
- Serial communication typically has a slower data transfer rate compared to parallel communication. This is because each bit is sent individually.
- Serial communication is generally simpler to implement due to the minimal wiring requirements.
- Example: USB, RS-232

Parallel Communication

- In parallel communication, multiple bits are sent simultaneously over multiple communication lines.
- Parallel communication requires more wires compared to serial communication.
- Parallel communication can achieve higher data transfer rates compared to serial communication.
- Parallel communication is more complex to implement.
- Example: Old Printer Ports

2. How instruction level parallelism is achieved in uniprocessor system?

- Instruction-level parallelism (ILP) refers to the ability of a processor to execute multiple instructions in parallel to improve performance.
- Even in a uniprocessor (single-core) system, there are techniques that can be employed to achieve instruction-level parallelism:

1. **Pipelining:**

- Pipelining is a technique where the execution of an instruction is divided into several stages, and multiple instructions can be in different stages of execution at the same time.
- Each stage performs a specific task, such as instruction fetch, decode, execute, and so on.
- This allows different stages of different instructions to be worked on simultaneously, increasing the throughput of the processor.

2. **Looping**

- This technique involves replicating loop bodies in the code, allowing multiple iterations of the loop to be executed in parallel.

(Asked 2 times)

3. What is Flynn's Taxonomy? Describe the different system illustrated by Flynn's Taxonomy in brief.

- Flynn's Taxonomy is a way to classify how a group of computers or processors work together when doing tasks.
- It's like sorting them into categories based on whether they do the same thing or different things and whether they work on the same data or different data.
- Each category has its own benefits and applications, and this classification helps computer designers choose the right architecture for specific tasks.
- There are four main groups in this classification:

1. Single Instruction, Single Data (SISD):

- This is the basic way how traditional computers used to work.
- One computer has one processor and does one thing at a time with one piece of information.
- The processor fetches one instruction, processes it, and then moves on to the next one in a sequence.

2. Single Instruction, Multiple Data (SIMD):

- In this group, one instruction is given, but many computers do the same thing to different pieces of information at once.
- It's like having a bunch of workers all doing the same task on different things.

3. Multiple Instruction, Single Data (MISD):

- This is rare and not commonly used.
- It's like having many computers doing different things to the same piece of information, which isn't very common in practice.

4. Multiple Instruction, Multiple Data (MIMD):

- It is the most flexible and commonly used category.
- This is when many computers each do their own thing with their own data.
- In MIMD systems, each processor independently executes its own set of instructions on its own data.
- Simply, they work independently on different tasks with different information.

(Asked 2 times)

4. Explain MIMD Architecture.

- It stands for Multiple Instruction, Multiple Data.
- It is the most flexible and commonly used category.
- In MIMD systems, each processor independently executes its own set of instructions on its own data.
- Simply, each processor works independently on different tasks with different information.
- It's a type of computer architecture where multiple processors or computing units work together to perform tasks.
- Each processor operates independently of the others.
- Processors can work on different tasks using different sets of instructions and data.
- MIMD systems aim to achieve parallelism, which means multiple tasks are executed simultaneously for better performance.
- Different processors can work on different types of tasks concurrently.
- This makes MIMD systems versatile and capable of handling a wide range of applications.
- Processors in a MIMD system can communicate with each other to exchange data and coordinate tasks.
- Communication enables synchronization and sharing of results.
- Examples: MIMD are used in Distributed System, Supercomputers, etc.

(Asked 2 times)

5. What is topology? Describe about different types of topologies used in multiprocessor communication.

- On microprocessor system, the topology defines a layout of how all the processors are interconnected to each other.
- The different types of topologies commonly used in microprocessor system are:

a. Bus Topology

- In a bus topology, all processors are connected to a single shared communication line, known as the "bus."
- Here, every piece of information passes through each processor.
- Not ideal for systems with high data transfer requirements.

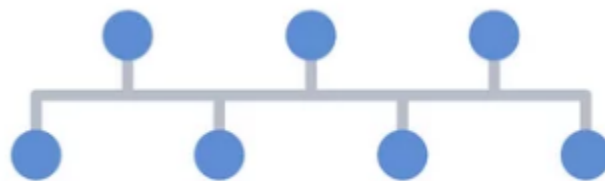


Fig: Bus Topology

b. Ring Topology

- It is a bus topology in a closed loop.
- In a ring topology, there is a single communication line which is connected to every processor. The last processor is connected with the first one.
- Data travels sequentially from one processor to the next until it reaches its destination.
- Efficient for systems where data flows in a linear pattern but can become problematic if a processor or link in the ring fails.

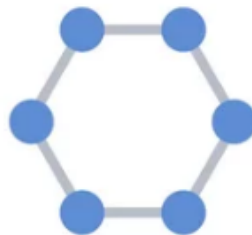


Fig: Ring Topology

c. Tree Topology

- In a tree topology (also known as a hierarchical topology), processors are organized in a hierarchical structure, like branches of a tree.
- Processors at higher levels are connected to lower-level processors, forming a tree-like structure.
- Provides a scalable way to connect a large number of processors, but the hierarchy can lead to longer communication paths.

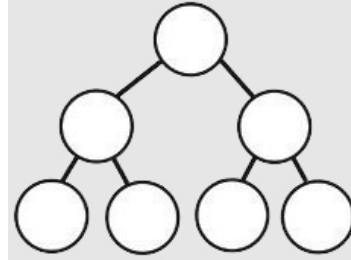


Fig: Tree Topology

d. Mesh Topology

- In a mesh topology, each processor is connected to every other processor in the system.
- Provides direct communication paths.
- It improves communication efficiency.
- Scalable, fault-tolerant, and suitable for systems with high intercommunication needs.



Fig: Mesh Topology

e. Hypercube Topology

- Hypercube topology extends mesh topology into higher dimensions.
- Each processor is connected to a unique set of other processors, forming a hypercube structure.
- Offers more direct communication paths, fault tolerance, and scalability as the dimensions increase.
- Complex to implement as dimensions increase.

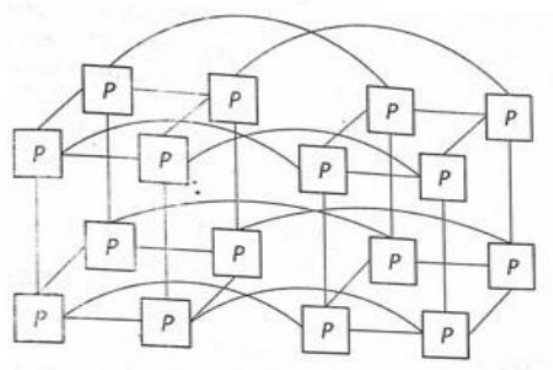


Fig: Hypercube topology

Q. Describe MIMD system topologies.

Q. What are the different topologies used to interconnect MIMD computers? Describe with suitable diagram.

Ans: MIMD system can be interconnected with the help of different topologies like Bus, Star, Tree, Mesh and Hypercube.

(Explain about MIMD and each topology with diagrams)

6. Explain communication organization strategies in multiprocessor system.

- In multiprocessor systems, where multiple processors work together, communication between processors is crucial for sharing data and coordinating tasks.
- Even the fastest processors cannot achieve high levels of efficiency if they spend too much of their time waiting to send and receive data.
- For this reason, multiprocessor systems use hardwired connections.
- Two common strategies for communication organization strategies are fixed connection and reconfigurable connection.

1. Fixed connections

- Fixed connection strategy establishes permanent communication paths between processors.
- Processors can only communicate with those directly connected to them.
- It's easier to set up as communication paths are predetermined.
- Changes in communication patterns require significant modifications.
- It is suitable for systems with stable and known communication patterns, and lower complexity requirements.

2. Reconfigurable connection

- Reconfigurable connection strategy allows processors to dynamically establish communication paths.
- Processors can communicate with other microprocessors based on current needs.
- It has a complex design.
- Suited for systems with changing communication patterns and scalability needs.

(Asked 5 times)

7. How is memory organized in multiprocessor systems? Illustrate with suitable diagram.

OR

Describe different memory organization techniques in multiprocessor systems.

Memory organization in multiprocessor systems refers to how multiple processors share and access memory resources.

- Shared memory architecture is used to organize memory in multiprocessor system.
- Shared memory is an architecture where multiple processors share a common memory, where all of them can read and write data.
- The UMA(Uniform Memory Access) and NUMA(Non-Uniform Memory Access) architecture both use shared memory.
- Through shared memory, the processors can access shared programs and data.
- They can also use the shared memory to communicate with each other via message passing.
- The first processor writes its message to the shared memory and signals the second processor that it has a waiting message.
- When the second processor is ready, it reads the message from shared memory.
- The location of the message in shared memory is either known beforehand or sent with the message waiting signal.
- In addition to message passing, the OS uses shared memory to store information about its current states.
- It is suitable for applications where communication and synchronization are important.
- Since processors often have their own caches, cache coherence protocols ensure that all caches have a consistent view of memory. When one processor writes to a memory location, other caches are updated accordingly.

Q. Explain UMA and NUMA with necessary diagrams.

UMA

- It stands for Uniform Memory Access.
- In a UMA architecture, all processors in a multiprocessor system share equal access to a single, shared main memory.
- This means that any processor can access any memory location with roughly the same access time.
- UMA requires robust cache coherence protocols to ensure that all processors have a consistent view of memory.

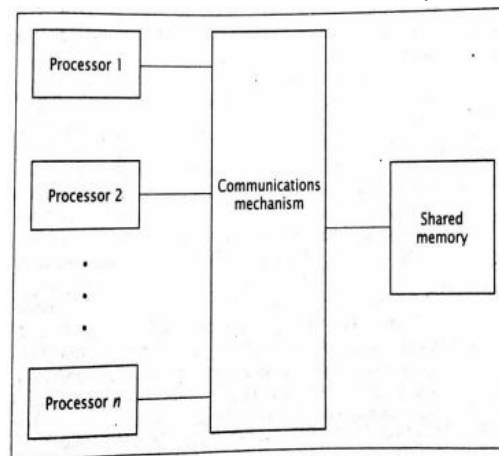


Fig: UMA

NUMA

- It stands for Non-Uniform Memory Access.
- It's an architecture designed to address the limitations of UMA as systems scale up in terms of processors and memory.
- In NUMA, the main goal is to reduce memory access latency by organizing memory into multiple banks, each associated with a subset of processors.
- The basic idea is to have each processor access its own local memory faster than remote memory, which leads to a more balanced memory access as the system scales.

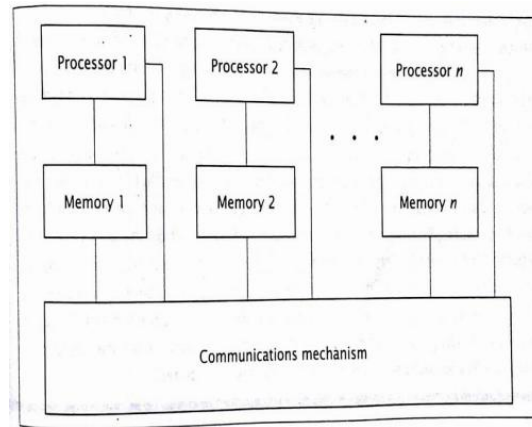


Fig: NUMA

(Asked 5 times)

8. Explain in brief about cache coherence. Describe how cache coherence can be dealt with.

- In a multiprocessor environment, all the processors in the system share the main memory via a bus.
- Now, keeping a common cache for all the processors will enhance the size of the cache thereby slowing down the performance of the system.
- For better performance, each processor implements its own cache.
- Cache coherence is a challenge that arises in multiprocessor systems, especially those with multiple processors or cores, each with their own cache memory.
- The problem occurs when multiple caches store copies of the same memory location, and changes to that location can lead to inconsistencies between the cached copies.
- This can result in incorrect or outdated data being used by different processors, leading to program errors and unexpected behavior.

- For example, consider two processors, P1 and P2, both caching the same data X from the main memory.
- If processor P1 updates the value of X, and before saving it to the memory P2 processor also operates on value X.
- Processor P2's cache might still have the old value, causing confusion and inconsistency.

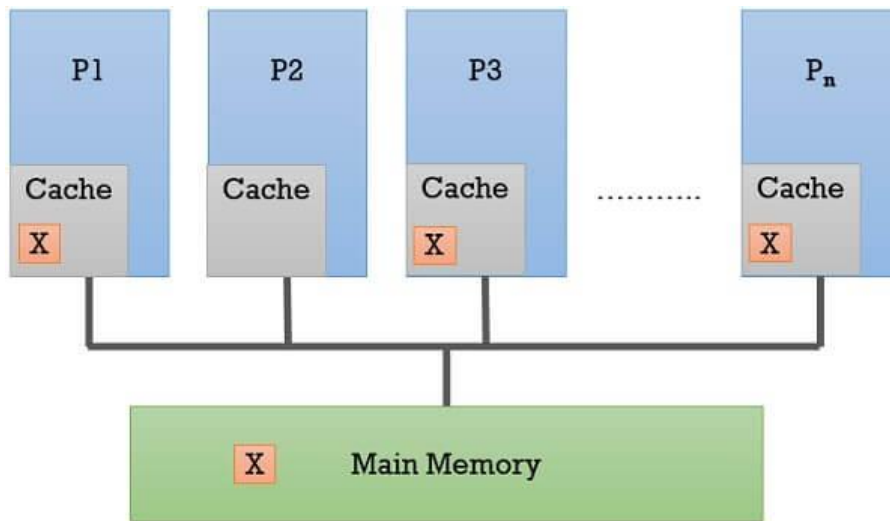


Fig: Processor sharing same memory

The cache coherence problem can be solved by various method which are explained below:

1. Snooping

- It involves processors monitoring or "snooping" the shared communication bus for memory operations initiated by other processors.
- When a processor writes to a memory location, snooping processors check if they have a copy of the same data in their cache.
- If another processor has the same data, the snooping processors are notified to update their copy.
- Snooping ensures that all caches have consistent and up-to-date data.
- It prevents different processors from having conflicting views of memory.

2. MESI protocol

- MESI is a widely used cache coherence protocol that helps manage the consistency of cached data in multiprocessor systems.
- It stands for Modified, Exclusive, Shared, and Invalid.
- Each cache line has one of these states, and the protocol determines how caches respond to read and write requests.

1. **Exclusive (E):** If a cache has a copy of memory that matches what's in main memory and hasn't been modified, it's in the Exclusive state. It's a clean copy.
2. **Shared (S):** If multiple caches have the same memory location and none of them have modified it, they're all in the Shared state. This keeps the data consistent.
3. **Modified (M):** If a cache has a copy of a memory location and it's been modified (changed), it's in the Modified state. The copy is different from what's in main memory.
4. **Invalid (I):** If a cache's copy is no longer valid because another processor modified the memory, the cache line is in the Invalid state. The cache must fetch the updated data when needed.

MESI protocol ensures that caches always have the correct and consistent data.

Q. Explain in brief about MESI protocol.